

LEASEOS

Contact, Communications & Emergency Directory

Implementation Build Plan for Claude

Purpose: add a privacy-preserving, tenant-scoped operational contact system to LeaseOS without importing workers' personal address books.

Core design rule

LeaseOS stores approved work relationships and job-scoped contact methods. It must not bulk-read, sync, or upload a worker's personal phone contacts.

Prepared for repository implementation and code review

Version 1.0 - September 19, 2026

Contents

1. Build objective and success criteria
2. Existing LeaseOS architecture alignment
3. Contact-domain architecture
4. Actors, directory layers and visibility
5. Data model
6. Privacy and permission model
7. Job contact roster and client projection
8. Historical relationship catalog
9. Emergency and ERAP directory
10. Mobile call handoff and device permissions
11. Offline-first behavior
12. API and service contracts
13. AI Secretary integration
14. Audit, provenance and lifecycle
15. Required UI surfaces
16. Staged implementation plan
17. Testing and release gates
18. Migration and rollout
19. Claude implementation instructions
20. Definition of done
- Appendix A. Suggested schemas
- Appendix B. Suggested endpoints
- Appendix C. Reference sources

1. Build objective and success criteria

Build one first-class LeaseOS contact service used by administration, dispatch, field drivers, safety, client portals, TDG/ERAP workflows, offline job packages and the AI Secretary. The subsystem is not a replacement for the device address book. It is an operational directory governed by organization, job assignment, role, contact-method classification and purpose.

- Company administration can maintain its workforce and operational directory: dispatch, safety, HR, payroll, mechanics, supervisors, managers, owner, drivers, labourers and other approved roles.
- Dispatch can assemble the exact contacts needed for a job, including client/consultant representatives, medic, disposal facility, site contacts, loading/unloading contacts, towing/recovery or other vendors.
- Drivers see contacts relevant to assigned work and approved historical relationships, not the entire corporate or client address book.
- Client users see only the assigned driver and explicitly published contractor contacts for that job, such as dispatch and designated emergency/safety contacts.
- Current-job emergency contacts remain available offline.
- Tapping a number hands the number to the device dialer; LeaseOS does not need to directly place calls in the initial implementation.
- Every sensitive read is authorized at the contact-method level and every important change is auditable.

Success test

A user must never receive a phone number merely because they are allowed to see the person. The requested contact method must independently pass tenant, role, job, classification and purpose checks.

2. Existing LeaseOS architecture alignment

This module should extend the existing LeaseOS design rather than create a parallel framework. The current project specification already establishes patterns that this module should reuse:

Existing LeaseOS rule	How contacts must align
Tenant-scoped application data	Every organization-owned contact, relationship, assignment and query is tenant-scoped unless it is an explicitly global LeaseOS-managed emergency/system record.
Roles & permissions are an explicit missing admin layer	Contact permissions should plug into the same role/permission service, not create a separate authorization system.
Append-only audit log	Contact edits, publication changes, role/visibility changes, emergency catalog revisions and overrides produce append-only audit events.
Genuine offline architecture is planned	Only authorized current/recent job contacts and emergency data are cached on the device; never mirror the entire organization directory.
Provenance is cross-cutting	System-managed emergency contacts and imported/entered records carry source, status, verification date and accountable verifier.
AI proposes; humans commit important values	The AI Secretary may locate/suggest contacts but may not silently publish private contact information or alter visibility.

Recommended integration point: implement the contact service after the shared role/permission foundation is available, then make dispatch, offline job packaging, emergency workflows and the AI Secretary consume it through a single service boundary.

3. Contact-domain architecture

LEASEOS CONTACT SERVICE

```

LeaseOS platform directory
Company directory
Partner / operations catalog
|
+--> Job contact roster --> Driver projection
|                               --> Client projection
|                               --> Office projection
|
+--> Relationship history
|
+--> Emergency catalog / ERAP binding
|
|   +--> Offline job package
|   +--> AI Secretary lookup
|   +--> System dialer handoff

```

Do not let screens query raw contact tables directly. Screens and agents request a projection from the contact service. The service owns authorization, field-level redaction, lifecycle status and audit behavior.

4. Actors, directory layers and visibility

Directory layer	Examples	Who may see it
LeaseOS platform directory	LeaseOS support, billing, platform safety/security escalation	Contracting-company admins and selected end-user support surfaces.
Company directory	Drivers, dispatch, mechanics, safety, HR, payroll, supervisors, management, owner	Internal users according to role and contact-method classification.
Partner / operations catalog	Clients, consultants, disposals, medics, shops, towing/recovery, vendors, terminals, loading facilities	Office/dispatch broadly; field users only when relevant to their work.
Job contact roster	Exact people and facilities assigned to one job	Users assigned to the job plus authorized client/office users.
Emergency catalog	911, CANUTEC, job ERAP holder/number, company emergency, site medic, local/emergency authorities as configured	Available according to emergency policy; current-job emergency data must be offline-capable.
Relationship history	Prior approved work contacts from completed jobs	Only users whose policy permits historical operational relationships; never a route to enumerate another organization.

Recommended role families

- platform_admin
- tenant_owner
- tenant_admin
- dispatcher
- safety
- hr
- manager
- mechanic
- driver
- labourer/operator
- client_admin
- client_job_viewer
- auditor/read_only

Do not hard-code these role names into contact logic. Authorization should check capabilities such as `contacts.company.read`, `contacts.job.read`, `contacts.client.publish`, `contacts.emergency.manage` and `contacts.private_employee.read`.

5. Data model

Separate identity, organization membership, contact methods and job assignment. This avoids duplicating a person when roles or companies change and prevents accidental exposure of every phone number attached to a visible person.

Entity	Purpose	Key fields
people	Human identity independent of a company	id, displayName, status, optional external identity refs

Entity	Purpose	Key fields
organizations	Tenant, client, consultant, facility operator, vendor or LeaseOS platform org	id, tenantId?, type, legal/display name, status
organization_memberships	Person-to-organization relationship	personId, organizationId, roleTitle, department, activeFrom/To, status
contact_methods	Individual phone/email/radio/other methods	owner type/id, method type, encrypted value, classification, status, verifiedAt, validFrom/To
organization_relationships	Tenant-to-client/vendor/consultant relationships	tenantId, externalOrganizationId, relationshipType, status
job_contact_assignments	Binds an approved contact or facility to a job with a role	jobId, subject, contactRole, visibility, effective times, source, addedBy
contact_access_policies	Optional data-driven policy overrides	scope, subject capability, classification, action, conditions
emergency_contacts	System or tenant emergency records	jurisdiction/scope, type, number, source, verifiedAt, status
job_erap_bindings	Manifest/shipping-document ERAP data for the job/load	job/load/manifest id, ERAP ref, telephone, sourceDocumentId, valid/status
contact_revisions	Historical validity of contact methods	contactMethodId, prior/new metadata, validFrom/To, supersededBy
contact_access_events	Security/audit telemetry for sensitive accesses/actions	actor, job, contactMethod, action, decision, timestamp, reason

6. Privacy and permission model

LeaseOS should classify each contact method. Classification is distinct from the person's role.

Classification	Meaning	Default exposure
public_work	Business contact intentionally usable across approved business relationships	Broadest work use, still tenant/purpose checked.
internal_work	Internal corporate work method	Same organization according to role/capability.
job_only	Only for active/authorized job participants	Job projection only.
client_visible	Approved for the client-side job roster	Only on jobs where explicitly published.
dispatch_only	Operational number for dispatch/office	Dispatch/management capabilities.
emergency_only	Use for emergency or designated emergency workflows	Emergency surfaces and authorized safety users.
private_personal	Sensitive personal number held for a narrow internal purpose	Never returned through field/client operational directories; tightly restricted HR/owner use only if required.

Hard privacy boundary

Do not use READ_CONTACTS to bulk-ingest device contacts. Do not copy a worker's personal address book into LeaseOS. If a future feature lets a user add a LeaseOS work contact to their phone, make it an explicit one-contact export through the system contacts UI.

Authorization decision

```
authorizeContactMethod(actor, contactMethod, purpose, job?)
1. authenticate actor
2. establish tenant boundary
3. load capability set
4. check contact-method classification
5. if job-scoped: verify actor/job relationship
6. if client user: apply client projection allowlist
7. if historical: verify prior assignment + history policy
8. if emergency-only: verify emergency policy/purpose
9. return ALLOW with redacted projection or DENY
10. write audit/access event where policy requires
```

Fail closed. Unknown role, missing tenant, stale assignment, absent policy, deleted contact method or ambiguous scope returns DENY or a redacted record - never a permissive fallback.

7. Job contact roster and client projection

The job contact roster is the central operational object. Dispatch assembles contacts when the job is created or modified. Some contacts can be auto-suggested from company defaults or relationship history, but publication is explicit and auditable.

Typical job role	Driver view	Client view	Office view
Assigned driver	Self/crew as needed	Yes - assigned driver only if published	Yes
Dispatcher	Yes	Yes if published	Yes
Company safety/emergency	Yes	Yes if published	Yes
Owner/manager	If operationally relevant	Only if explicitly published	Yes by role
Client representative/company man	Yes when assigned	Own authorized contacts	Yes
Consultant	Yes when assigned	As permitted	Yes
Medic/site emergency	Yes	As permitted	Yes
Disposal/loading facility	Yes	Only if client policy permits	Yes
Mechanic/shop	When assigned/relevant	No by default	Yes
HR/payroll	No operational exposure	No	Role-limited internal only

Roster rules

- Never publish an entire company directory to a job.
- Client-side contacts are an explicit projection, not the same payload as the internal job roster.

- When an assignment ends, active-job visibility ends automatically unless a historical-work policy separately allows the contact.
- Removal from a roster does not erase history; preserve a revision/audit trail.
- Job packages should snapshot enough historical identity/contact metadata to explain what was used at the time without re-exposing data later.

8. Historical relationship catalog

Drivers need a catalog of approved contacts from jobs they personally worked, so repeat work with the same consultant/client/facility is easier. This is relationship history, not unrestricted directory search.

- Index by driver + organization + job participation.
- Show role, organization, last worked date, number of shared jobs and last verification date.
- Only return contact methods still active and permitted by the current classification/policy.
- If a method is stale or superseded, label it and prefer the current verified method if policy allows.
- Never allow history search to enumerate all members of a client/consulting company.
- Dispatch may use history as a suggestion source when creating a new job roster; a human confirms the assignment.

Example UI:

```
ABC Consulting
  John Smith - Field Consultant
  4 shared jobs | last worked 2026-04-12
  Work mobile - verified 2026-04-01
  [Open history] [Dial] [Add to current job - dispatcher only]
```

9. Emergency and ERAP directory

Emergency data needs stronger provenance and availability than ordinary contacts. Model two sources: (1) LeaseOS/system-managed official emergency records and (2) tenant/job-supplied emergency contacts. Keep source, jurisdiction/scope, verification date and status on every record.

ERAP behavior

- Do not treat an ERAP telephone number as a universal catalog constant.
- When a shipment requires an ERAP, bind the ERAP reference number and ERAP telephone number from the applicable shipping document/manifest to the job/load emergency roster.
- Preserve sourceDocumentId and the exact effective value used for the trip/job.
- If the manifest changes, create a new binding/revision rather than silently overwriting the prior value.
- Only surface an ERAP number for the load/manifest to which it applies.

System emergency seed records

At minimum, seed 911 for local emergency calling and the current Transport Canada CANUTEC numbers as system-managed records, with source metadata and a scheduled verification process. As of this build-plan date, Transport Canada lists 1-888-CAN-UTEC (226-8832), 613-996-6666 and *666 on a cellular phone in Canada for dangerous-goods incidents. Keep these values configurable and source-verified rather than permanently hard-coded in application logic.

Emergency UX rule

Emergency access must be simple, highly visible and offline-capable, but emergency visibility does not erase tenant/job privacy boundaries for unrelated personal numbers.

10. Mobile call handoff and device permissions

For Android, the preferred initial behavior is to hand the selected number to the system dialer. Use ACTION_DIAL with a tel: URI. The dialer displays the number and the user confirms the call. Do not use ACTION_CALL unless a future, separately justified requirement needs direct calling.

```
// Android / Kotlin example
val intent = Intent(Intent.ACTION_DIAL).apply {
    data = Uri.parse("tel:${authorizedPhoneNumber}")
}
startActivity(intent)
```

- No READ_CONTACTS permission for this feature.
- No bulk address-book import.
- No CALL_PHONE permission required for ACTION_DIAL.
- Log the in-app dial action if appropriate, but do not attempt to capture call audio or content.
- If a number is hidden/redacted by policy, never place it in the Intent, logs, analytics or crash metadata.

11. Offline-first behavior

This subsystem must integrate with LeaseOS's genuine offline plan. The contact cache is a projection, not a database replica.

Cache on device	Do not cache broadly
Current assigned-job contacts	Entire company directory
Current-job client/consultant/medic/disposal contacts	Unrelated client organizations
Required company dispatch/safety contacts	HR/payroll/private employee records
Current-job ERAP binding and approved emergency directory	Historical contacts the user is no longer authorized to access
Limited recent relationship history if enabled by policy	All partner/vendor records

- Encrypt local contact data using the same secure local-storage strategy as other LeaseOS offline records.
- Include authorization version / assignment fingerprint in cached packages.
- On reconnect, invalidate removed assignments and stale contact methods promptly.
- Do not silently merge conflicts in visibility/classification. Surface them to an authorized human.
- Offline emergency view must clearly show last-sync/verification timestamps where relevant.

12. API and service contracts

Prefer narrow projection endpoints over raw CRUD exposure. All endpoints require authenticated tenant context unless explicitly global/system-managed.

Service area	Suggested operations
Company directory	contacts.company.list/search/get; contacts.memberships.create/update/end; contacts.methods.create/update/supersede
Partner catalog	contacts.partners.list/search/get; contacts.relationships.create/update
Job roster	contacts.job.list; contacts.job.assign; contacts.job.remove; contacts.job.publishToClient; contacts.job.suggest
History	contacts.history.byOrganization; contacts.history.byPerson; contacts.history.forCurrentUser
Emergency	contacts.emergency.forJob; contacts.emergency.systemCatalog; contacts.emergency.tenantCatalog; contacts.erap.bindFromManifest

Service area	Suggested operations
Dial action	contacts.resolveDialTarget - returns authorized tel payload or deny reason
Offline	contacts.offline.packageForAssignments; contacts.offline.delta
Admin policy	contacts.policies.list/update; contacts.classifications.list
Audit	audit.list filtered by contact entities and access events

Projection principle

```
// Bad
GET /people/:id -> returns every phone/email field

// Good
contacts.resolveProjection({
  actorId,
  subjectId,
  jobId?,
  purpose: 'job_roster' | 'dispatch' | 'client_view' | 'emergency' | 'history'
})
-> { displayName, organization, role, allowedMethods[], redactions[] }
```

13. AI Secretary integration

The AI Secretary must consume the same contact service as every other client. Do not grant the model raw-table or cross-tenant directory access.

- Provide typed tools such as findJobContact(jobId, role), findDisposalContact(jobId), findCompanyDispatch(), findEmergencyContact(jobId, type).
- The tool layer enforces actor identity and authorization before returning fields to the model.
- The AI can suggest adding a known historical contact to a job, but dispatch/human confirmation is required to publish it.
- The AI may say that a contact is unavailable or restricted; it must not infer or fabricate a number.
- When asked to call, the AI should resolve the authorized target and open the dialer - not place the call silently.
- Agent prompts/logs should avoid retaining private_personal values unless absolutely required for the specific authorized task.

14. Audit, provenance and lifecycle

Contact information changes. The design must preserve both current usability and historical accountability.

Event	Required record
Create/update contact method	actor, tenant, source, classification, old/new metadata, reason, timestamp
Supersede phone/email	prior method validTo, successor link, verifier/source
Publish to client	who published, which job, which method, effective period
Assign/remove from job	actor, job, role, source/suggestion provenance
Emergency catalog update	official/tenant source, verifier, verification timestamp

Event	Required record
ERAP binding change	manifest/shipping document reference, old/new binding, reason
Sensitive access/dial	actor, contact method id, purpose, job if applicable, allow/deny decision as policy requires

Recommended status vocabulary: active, inactive, superseded, pending_verification, rejected, unknown. Unknown or unverified data must not be visually indistinguishable from verified data.

15. Required UI surfaces

Surface	Minimum content
Admin > Company Directory	People/memberships, departments, work methods, classifications, status, verification, role permissions.
Office > Partner Contacts	Clients, consultants, facilities, medics/vendors, relationship type, verified methods.
Dispatch > Job Contacts	Roster builder, suggestions, role labels, visibility chips, publish-to-client controls, ERAP/emergency section.
Field > Contacts	Current job roster, company dispatch/safety, limited history, tap-to-dial.
Field > Emergency	911, CANUTEC, job ERAP, company emergency, medic/site/client emergency; offline status.
Client Portal > Job Contacts	Assigned driver plus only explicitly client-visible contractor contacts.
History > Worked With	Organizations/people tied to prior authorized jobs with last-worked and verification information.
Admin > Contact Policy & Audit	Classification policy, access/audit events, emergency catalog verification, stale-contact queue.

16. Staged implementation plan

Build in vertical slices. Each phase must leave the repository green. Do not build all UI first or all schemas first without proving the authorization path end-to-end.

Phase C0 - Repository reconnaissance and contract freeze

Inspect current schema, routers, auth middleware, tenant context, jobs, users, facilities, offline work and auditLog implementation. Identify existing person/user/org concepts before adding tables. Write a short mapping of reused vs new entities. No parallel auth or tenant system.

Phase C1 - Contact domain + authorization foundation

Add/extend people, organization memberships, contact methods, classification enum and contact service. Implement field-level authorization. Add migrations, indexes, tenant constraints and unit tests for every classification.

Phase C2 - Company directory vertical slice

Admin creates a worker work-phone contact; dispatcher can retrieve it if permitted; driver cannot retrieve restricted/private method. Add UI for internal directory and verification/status.

Phase C3 - Job contact roster + client projection

Add job_contact_assignments and roster service. Dispatch assigns driver/consultant/medic/disposal/dispatch contact. Prove separate driver, office and client payloads from the same roster.

Phase C4 - Emergency catalog + ERAP binding

Implement system/tenant emergency records, 911/CANUTEC seed data with provenance, manifest-linked ERAP binding, emergency screen and stale-source verification controls.

Phase C5 - Relationship history

Build prior-job index/projection. Show only contacts connected to the current user's permitted job history. Add stale/superseded logic and dispatch suggestion workflow.

Phase C6 - Android dialer handoff

Resolve authorized dial target server/local-policy side, then launch ACTION_DIAL. Explicitly verify manifest does not request READ_CONTACTS or CALL_PHONE for this workflow.

Phase C7 - Offline contact package

Add contact projection to pre-departure cache and sync/delta logic. Encrypt locally, bind to assignment fingerprint, invalidate revoked access, test no full-directory replication.

Phase C8 - AI Secretary tools

Expose narrow typed contact lookup tools behind the same policy service. Add tests proving the model cannot enumerate cross-tenant or private methods.

Phase C9 - Hardening, audit and pilot readiness

Stale-contact review queue, emergency-source verification, permission regression suite, tenant isolation tests, audit UI, telemetry, backup/restore behavior and production migration rehearsal.

17. Testing and release gates

Mandatory automated test families

- Tenant isolation: tenant A cannot discover tenant B people, methods, relationships, history or job rosters by ID guessing, search, pagination, export or AI tool.
- Classification matrix: every role/capability against every classification for read, dial, publish and edit.
- Job assignment: assigned vs unassigned driver, active vs completed vs cancelled job, client vs internal projections.
- Historical relationship: prior participant allowed only within configured policy; unrelated coworkers denied.
- Client projection: never returns HR, payroll, unrelated mechanics/drivers, internal-only methods or other clients.
- Emergency: current job ERAP appears only when bound/applicable; revision preserves old binding; offline package contains required emergency set.
- Mobile privacy: no READ_CONTACTS dependency; ACTION_DIAL path; denied/redacted numbers never appear in intents/logs.
- Offline: revoked assignment invalidates cached access; stale data visibly marked; no automatic visibility conflict merge.
- Audit: important create/update/supersede/publish/assignment events are append-only and attributable.
- AI tools: no raw directory enumeration, cross-tenant leakage, hidden-field leakage or fabrication when contact is unavailable.

Release gate

Gate must be green

Company admin creates a work contact -> dispatch assigns it to a job -> assigned driver sees only allowed methods -> client sees only published subset -> driver taps Dial and system dialer opens -> offline job package preserves authorized emergency/job contacts -> assignment is revoked and access disappears -> audit history remains intact.

18. Migration and rollout

1. Inventory existing user/driver/client/facility phone/email fields and classify each source before migration.
2. Create canonical contact methods without deleting legacy fields immediately.
3. Dual-read during a short compatibility period if necessary; new writes go through the contact service only.
4. Migrate company work contacts first. Do not automatically promote arbitrary legacy personal numbers to operational/client-visible classifications.
5. Require admin review for ambiguous employee numbers and all client-visible defaults.
6. Migrate facilities/disposal contacts with organization ownership and source/verification metadata.
7. Enable job rosters for a pilot tenant, then emergency/offline/history in controlled phases.
8. After parity and audit checks pass, remove deprecated direct phone/email reads from old modules.
9. Add a structural test that fails if protected contact tables are queried outside the approved repository/service boundary.

19. Claude implementation instructions

Use this as the implementation brief

Claude should inspect the repository before coding, reuse existing LeaseOS conventions, implement in small green commits, and stop/fail closed when a required authorization or provenance rule cannot be established.

CLAUDE HANDOFF

Implement the LeaseOS Contact, Communications & Emergency Directory described in this document.

Non-negotiable rules:

1. Do not create a parallel tenant, auth, RBAC, audit or offline framework. Reuse the canonical LeaseOS services.
2. Do not bulk-read, import, sync or upload users' device address books.
3. A visible person does NOT imply all of their contact methods are visible. Authorize each method separately.
4. Enforce tenant + capability + job relationship + classification + purpose. Fail closed.
5. Client job views are an explicit allowlisted projection, never the internal roster payload.
6. Historical contacts are only from authorized prior job relationships; history must not become directory enumeration.
7. ERAP numbers are job/load/manifest data when applicable. Preserve shipping-document source and revisions.
8. Use ACTION_DIAL for Android call handoff in the first implementation; do not request READ_CONTACTS or CALL_PHONE for this flow.
9. Offline cache only authorized projections needed for current work; never mirror the full directory to a field device.
10. AI Secretary gets narrow typed contact tools behind the same authorization service; no raw-table contact access.
11. Append audit/provenance records for important changes and publication decisions.
12. Add tests before/with every authorization rule and prove negative cases, tenant isolation and client redaction.

Implementation sequence: C0 -> C1 -> C2 -> C3 -> C4 -> C5 -> C6 -> C7 -> C8 -> C9.

After each phase: run typecheck, lint, unit/integration tests and the repository's canonical full gate; record the commit hash and test count.

Do not mark a phase DONE unless its end-to-end gate is proven.

20. Definition of done

- One canonical contact service is used by admin, dispatch, field, client, emergency, offline and AI flows.
- No feature requires broad access to a device personal address book.
- Contact-method classifications and capability checks are enforced server-side and, where needed for offline mode, through signed/versioned local policy data.
- Job rosters support internal and client-safe projections.

- Past-job relationship lookup works without enabling organization-wide enumeration.
- ERAP data is bound from applicable job/shipping-document context and emergency contacts have source/verification metadata.
- Android dialer handoff works without READ_CONTACTS/CALL_PHONE in the initial design.
- Offline current-job contacts and emergency numbers work with no network connection.
- Assignments/revocations update access correctly after sync.
- Append-only audit/provenance exists for important contact changes and publications.
- Negative authorization tests, cross-tenant tests and AI-tool leakage tests are green.
- Repository full gate is green and implementation evidence is recorded.

Appendix A. Suggested schemas

// Illustrative only - adapt to existing Drizzle conventions and IDs

```
contactMethods {
  id
  tenantId?
  ownerType: 'person' | 'organization' | 'facility'
  ownerId
  methodType: 'mobile' | 'phone' | 'email' | 'radio' | 'other'
  valueEncrypted
  displayValue? // optional safe formatted projection, not a second source of truth
  classification: 'public_work' | 'internal_work' | 'job_only' |
    'client_visible' | 'dispatch_only' | 'emergency_only' |
    'private_personal'
  status: 'active' | 'inactive' | 'superseded' | 'pending_verification' | 'rejected'
  source
  verifiedAt?
  verifiedBy?
  validFrom
  validTo?
  supersededBy?
}

jobContactAssignments {
  id
  tenantId
  jobId
  subjectType: 'person' | 'organization' | 'facility'
  subjectId
  contactRole
  internalVisible: boolean
  clientVisible: boolean
  emergencyRelevant: boolean
  effectiveFrom
  effectiveTo?
  source
  addedBy
}

jobErapBindings {
  id
  tenantId
  jobId
  manifestId?
  loadId?
  erapReferenceNumber
  erapTelephone
  sourceDocumentId
  status
  effectiveFrom
  effectiveTo?
  verifiedAt?
  verifiedBy?
}
```

```

}

contactAccessEvents {
  id
  tenantId?
  actorId
  jobId?
  contactMethodId?
  action: 'read' | 'dial' | 'publish' | 'edit' | 'export'
  purpose
  decision: 'allow' | 'deny'
  reasonCode
  occurredAt
}

```

Appendix B. Suggested endpoint / tool list

```

contacts.company.list
contacts.company.search
contacts.person.getProjection
contacts.methods.create
contacts.methods.update
contacts.methods.supersede
contacts.memberships.create
contacts.memberships.update
contacts.memberships.end

contacts.partners.list
contacts.partners.search
contacts.relationships.create
contacts.relationships.update

contacts.job.list
contacts.job.assign
contacts.job.remove
contacts.job.suggest
contacts.job.publishToClient
contacts.job.clientProjection

contacts.history.forCurrentUser
contacts.history.byOrganization

contacts.emergency.forJob
contacts.emergency.systemCatalog
contacts.emergency.tenantCatalog
contacts.erap.bindFromManifest

contacts.resolveDialTarget
contacts.offline.packageForAssignments
contacts.offline.delta

```

Appendix C. Reference sources

External sources used to validate the privacy, Android and ERAP portions of this plan. Repository implementation must still confirm the current law/platform requirements at release time.

- [Transport Canada - Emergency preparedness, response and ERAP FAQ](#)
- [Transport Canada - Emergency Response Assistance Plans \(ERAPs\)](#)
- [Transport Canada - Guide for persons who have an ERAP](#)
- [Alberta - Collecting personal information under PIPA](#)
- [Alberta - Protecting personal information under PIPA](#)
- [Android Developers - Common intents: phone calls](#)
- [Android Developers - Minimize permission requests](#)
- [Android Developers - Manifest.permission.READ_CONTACTS](#)